Learning Feature-Based Heuristics for Approximate Longest Paths

Marc Abou Serhal mza23@mail.aub.edu
Yasmina Mansour ysm12@mail.aub.edu
Fatima Bazzoun fmb26@mail.aub.edu

November 2025

Introduction

The LONGEST PATH problem is a fundamental topic in computer science and mathematics, playing an important role in understanding how complex systems can be explored, optimized, and structured. At a high level, the problem captures the challenge of finding a sequence of steps that explores as much of a system as possible without revisiting the same state. Because of this, it naturally arises in many real-world applications where efficiency, coverage, and resource utilization are critical. Some of its key applications include:

- **Scheduling problems**: arranging tasks in an efficient sequence without repetition.
- **Route and travel planning**: planning efficient delivery, robotics path planning, etc.
- **Logistics**: optimizing vehicle movement in order to minimize cost and time.
- **Computer chip design**: optimizing how circuit components are connected can minimize the total wire length, reduce signal delay, and lower power consumption.
- **DNA sequencing and genome assembly**: arranging fragments of DNA in a sequence that reconstructs the original strand.

Despite its wide range of applications, finding the longest path in a general graph is computationally hard; in particular, deciding whether a graph contains a path of length at least k is NP-complete. As a result, exact solutions are often infeasible for large instances. Nevertheless, many practical systems rely on approximations, heuristics, and problem-specific insights inspired by the longest path problem to achieve good performance in practice.

The goal of this project is to leverage machine learning to find approximate longest paths in graphs. By training a model to learn effective heuristics from data, the model can guide the path-finding process toward high-quality solutions, even when exact computation is infeasible.

Model

The model will take as input local and global structural features of a vertex u in a graph G, and predict the length of the longest path in G starting from u, denoted $\ell_G(u)$.

SCC-Level Features

Each vertex u belongs to some strongly connected component (SCC) $C(u)$. We will compute the condensantion graph G_C of G (it is the directed acyclic graph where each vertex represents a SCC of the original graph, each being labeled with its original size) and calculate the following:

(a) The size of $C(u)$, denoted $|C(u)|$.

(b) The number of edges between vertices of $C(u)$, denoted $|E^-_{C(u)}|$.

(c) The number of edges from vertices in $C(u)$ to other SCCs, denoted $|E^+_{C(u)}|$.
This value indicates how well connected $C(u)$ is to other SCCs, and the freedom a path has in exiting it.

(d) The largest sum of a path in G_C starting from $C(u)$, denoted $U(u)$.
This value is an upper bound on $\ell_G(u)$, as in the best case, a path will visit each vertex in a SCC before exiting it and entering the next one.

(e) The longest DFS path found within $C(u)$, denoted $\ell_{\text{DFS}}(C(u))$.

Features (a), (b) and (e) indicate the time u can remain in $C(u)$ before exiting/halting.

Vertex-Level Features

Using the both the original graph and G_C, we calculate the following:

(f) The number of outgoing edges from u to vertices in $C(u)$, denoted $out^-(u)$.

(g) The number of outgoing edges from u to vertices outside $C(u)$, denoted $out^+(u)$.

(h) The number of ingoing edges into u from vertices in $C(u)$, denoted $in^-(u)$.
Note that the number of ingoing edges into u from vertices outside $C(u)$ is an irrelevant feature as such edges can never be used when starting from u.

(i) The length of the longest path starting from u using DFS paths to exit vertices (vertices with edges to other SCCs), denoted $L(u)$.
This value is a lower bound on $\ell_G(u)$, as it's a valid path starting from u, and it can be calculated via dynamic programming:

$$L(u) = \max_{v \in C(u)} \left(d_{\text{DFS}}(u, v) + \left(\max_{(v,w) \in E(G), w \notin C(u)} L(w) \right) \right)$$

where $d_{\text{DFS}}(u, v)$ is the length of a DFS path from u to v.

(j) The length of the first DFS path to an exit vertex achieving $L(u)$, denoted $L_0(u)$.
Features (a), (b) and (e) indicate how pesimistic $L_0(u)$ is, and therefore how much better $\ell_G(u)$ is from $L(u)$.

Features (f), (g) and (h) are classic heuristics used in the backtracking algorithm for longest path.

Architecture

We will use a neural network that takes as features the 10-tuple:

$$x^{(u)} := \big(|C(u)|, |E^{-}_{C(u)}|, |E^{+}_{C(u)}|, U(u), \ell_{\text{DFS}}(C(u)), out^{-}(u), out^{+}(u), in^{-}(u), L(u), L_0(u)\big)$$

and predicts an approximate value $\widehat{\ell}_G(u) = \mathbb{E}[\ell_G(u)]$. To make use of the fact that the features include a lower bound $L(u)$ and an upper bound $U(u)$ of $\ell_G(u)$, a value in $[0, 1]$ is predicted and then normalized to fit in the range $[L(u), R(u)]$:

$$\widehat{\ell}_G(u) := \big(U(u) - L(u)\big)\sigma\big(\phi(x^{(u)})\big) + L(u)^{1}$$

where $\phi(x^{(u)})$ is the output of the second-to-last layer of the network.

Algorithm

The following algorithm finds an approximate longest path in a given directed graph G, which starts at the vertex with the longest expected path starting from it, and then iteratively removes the current vertex and moves to the neighbor with the longest expected path starting from it:

Algorithm 1: Path-finding algorithm

Input: A directed graph G
Output: A path $(u_1, ..., u_k)$ in G

$S \leftarrow V(G)$
// S is the set of eligible vertices to be chosen as the next vertex in the path
$i \leftarrow 0$
// i is the length of the current path
while $S \neq \emptyset$ **do**
  $i \leftarrow i + 1$
  Compute features $x^{(u)}$ for every $u \in V(G)$
  $u_i \leftarrow \arg\max_{v \in S} \widehat{\ell}_G(v)$
  // u_i maximizes the expected length of the longest path starting from it
  $S \leftarrow N_G(u_i)$
  // $N_G(u)$ is the set of neighbors of u in G
  $G \leftarrow G - u_i$
  // u_i has been used so it's removed from the graph
end
return $(u_1, ..., u_i)$

The above implementation is greedy as it only takes the expected best next vertex without considering other choices; the algorithm can be improved by using beam search (Table 1) or backtracking once there are no more eligible vertices to be chosen as the next vertex.

It's also important to note that the features need not be computed for vertices outside the SCC of the previously chosen vertex, as they don't change at its removal from the graph.

[1] $\sigma(x) = \dfrac{1}{1 + \exp(-x)}$ is the sigmoid function, however any function ranging in $[0, 1]$ also works.

Data Generation and Visualization

We generated 100 random graphs of size 26 with varying density and extracted many examples from those graphs, by running the feature extraction algorithms and the $\mathcal{O}(|E(G)|2^{|V(G)|})$ algorithm for determining longest paths.

We took advantage of the fact that, given a graph G, the algorithm in fact finds the longest path starting from every vertex, in every induced subgraph[2] of G. More specifically, it computes:

$$\text{DP}[u, S][3] = \begin{cases} \textit{true} & \text{if there's a hamiltonian path in } G[S] \text{ starting at } u \in S \\ \textit{false} & \text{otherwise.} \end{cases}$$

and so we get that $\ell_{G[S]} = \max\{|S'| : S' \subseteq S, \text{DP}[u, S']\}$. By computing these values using $\mathcal{O}(n2^n)$ DP and considering every induced subgraph $G[S]$ where $u \in S$ can reach all the vertices, we get many examples from u, each one restricting the set of usable vertices.

At the end, we ended up with over 50,000,000 examples, which we then truncated to approximately 100,000 examples to reduce the training time.

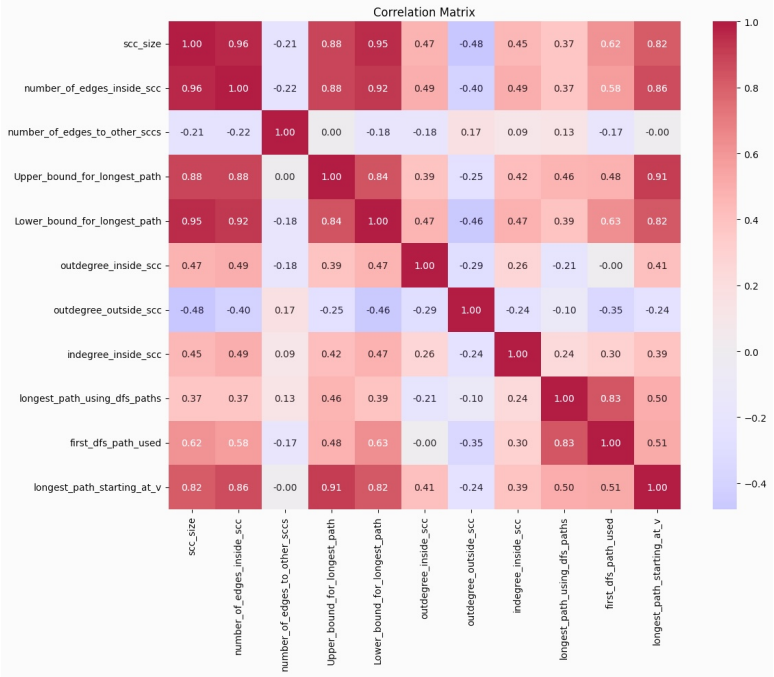

Figure 1: Correlation matrix between features of $x^{(u)}$ and $\ell_G(u)$

[2] $G[S] = (S, \{(u, v) \in E(G) : u, v \in S\})$ is the subgraph of G induced by S, which is the subgraph restricted to vertices in S and edges between them.

[3] Bitsets are used, which store each value as a single bit instead of a full byte, reducing memory usage by a factor of 8.

Training and Performance

Below are a few test statistics for the neural network, as well as performance metrics for both the algorithm ran with beam width 3, as well as the baseline heuristic of choosing the vertex with the biggest number of unvisited neighbors ran from all starts[4], which we ran on 100 held-out graphs of size 30 with around 60 edges each and average longest path length of 21.19.

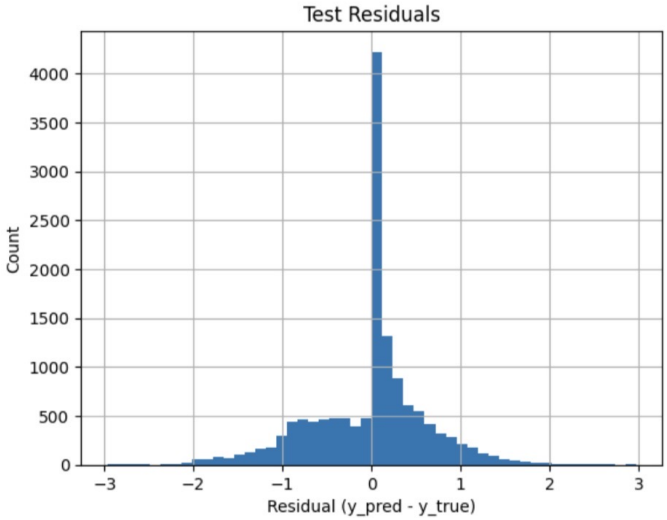

Figure 2: Test residuals for the neural network which achieved a MSE of 1.661

Metric	Statistic	Value (Algorithm)	Value (Baseline heuristic)
Performance ratio $\frac{\widehat{\ell}(G)}{\ell(G)}$	Average	0.898	0.758
	Minimum	0.731	0.462
Absolute error $\ell(G) - \widehat{\ell}(G)$	Average	2.31	5.180
	Maximum	7	14
Relative error $\frac{\ell(G)-\widehat{\ell}(G)}{\ell(G)}$	Average	0.102	0.242
	Maximum	0.269	0.538

Table 1: Performance metrics for both the algorithm ran with beam width 3, as well as the baseline heuristic ran from all starts. $\ell(G)$ is the longest-path length in G, $\widehat{\ell}(G)$ is the found-path length.

On average, our algorithm found a path of length 120% of the length of path found by using the baseline heuristic from every start, with an average of 2.87 more vertices.

[4]To account for the added complexity of extracting all the features in $x^{(u)}$, the baseline heuristic was run starting from every vertex in each graph.